



PROJECT-BASED LEARNING

PHASE-I EVALUATION

ExamSync – Smart Examination Coordination Portal

Team ID: DSCPP-III-2026-T079

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T079

Semester: 3rd

Section: D, DS1, A, E

Domain: DSA & OOPs

Team Members

1. Shreeya Sharma – 2510011488
2. Inakshi Bahukhandi – 2512520002
3. Rincy – 2510010306
4. Saumya Jha – 2510011508

Mentor

Mr. Kanishka Bhatt

Interactions completed : 3

Problem Statement & Motivation

Problem Statement

The problem is to develop a smart examination coordination system that reduces the difficulties and time involved in manual seating allocation, Searching examination related details through common lists, teacher duties, and UFM record keeping.

Motivation

- ▶ Searching examination related details through common lists takes time.
- ▶ Manual handling of the UFM case.
- ▶ Changes in student, room, or examination details can make seating arrangements difficult to update.

Target Users

- ▶ **Students:** Instant seating and room details.
- ▶ **Faculty:** Personalized duty and UFM access.
- ▶ **Admins:** Dynamic seating allocation, teacher duty allocation, and data management.

Objectives

Project Objectives

1. Automate seating allocation and minimize human errors.
2. Provide instant seat details to students using their roll number.
3. Manage teacher invigilation duties efficiently through the portal.
4. Digitally manage UFM records for organized examination handling.
5. Apply DSA and OOP concepts for efficient data management and system implementation.

Key Objective: Build a smart examination coordination portal that eliminates seating errors and ensures conflict-free student allocation

Proposed Solution

Proposed Approach

1. **Input:** Student, teacher, and room capacity data.
2. **Processing:** DSA for smart seat allocation and teacher duty allocation.
3. **System Logic:** OOPs, constraint validation and role-based access rules.
4. **Storage:** Hashing, dynamic structures.
5. **Visualization:** Role-based dashboards.

Key Features

- ▶ Conflict-free seating generator.
- ▶ Roll number based student portal.
- ▶ Teacher Duty Allocation & Personalized Duty Access
- ▶ Digital UFM incident logging.
- ▶ Dynamic room allocation manager.

Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures in C	Min-Heap, Doubly Linked List, MergeSort, Arrays, Hashing, Searching, Hash Table & Constraint Checkers	Seating and faculty duty allocation
OOPs with C++	Classes & Encapsulation, Inheritance & Polymorphism, Operator Overloading, Constructors/Destructors, Exception Handling, File I/O	System Architecture & I/O Layer

System Architecture / Workflow



Major Components

- ▶ **UI Layer:** Role-based access portals.
- ▶ **Logic Engine:** Constraint checker & allocation logic.
- ▶ **Data Management:** In-memory hash maps & SQL storage.

Data / Control Flow

- ▶ Admins input room and schedule parameters.
- ▶ Core engine processes conflict rules to map seats.
- ▶ Portal displays seating allocation for students and assigned duties for the teachers.

Technology Stack & Methodology

Technology Stack

- ▶ **Programming:** DSA and OOPs implementation.
- ▶ **Database:** Student, Exam, Room and Faculty record.
- ▶ **IDE & Compilers:** Visual Studio Code, GCC/G++
- ▶ **Version Control:** Git / GitHub

Development Methodology

1. Data of Students, Faculty, and room capacity.
2. OOP classes for Students, Faculty, and ExamHalls.
3. Implemented seating and duty allocation algorithms.
4. Testing for seating overlap prevention, room capacity constraints, and role-based permissions.
5. Linked DSA core engine with OOP controllers and SQL Server to get live student and teacher dashboards.

Initial Progress & Team Contribution

Progress Achieved

- Requirement analysis completed.
- Architecture prepared.

Individual Contribution

Member	Role	Contribution
Shreeya Sharma	DSA / Core Developer	Seating and faculty allocation, student record sorting, searching and dynamic seat mapping generation.
Inakshi Bahukhandi	DSA / Core Developer	capacity validation, seating constraints, faculty duty constraints, and conflict checking.
Rincy	Frontend & OOP Developer	C++ class hierarchy design, system controller logic, user interface design, and frontend flow testing.
Saumya Jha	Backend & Documentation	Backend module development, Server integration, and technical project documentation.

Roadmap, Expected Outcomes & References

Roadmap

1. **Phase-I:** Proposal & Architecture Design
2. **Phase-II:** Core Module Implementation
3. **Phase-III:** Integration, UFM & UI Portal

Expected Outcomes

- ▶ Automated seating and teacher allocation.
- ▶ Personalized student & teacher dashboards.
- ▶ Paperless UFM reporting system.

References

1. Ayeesha Begum S, et al. *Algorithm Based Seating Arrangement System*, IJFMR.
2. Asif A, et al. *Smart Exam Management Platform*, IJRTI.
3. Herbert Schildt, *C++: The Complete Reference*, McGraw Hill.
4. Karumanchi N., *Data Structures and Algorithms Made Easy*.